

```
# Import PyTorch library (used mainly for deep learning & GPU handling)
import torch

# Import TensorFlow library (deep learning framework)
import tensorflow as tf

# Import Pandas for data manipulation (CSV, tables)
import pandas as pd

# Import NumPy for numerical computations
import numpy as np

# Import OpenCV for image processing tasks
import cv2

# Import Google Colab drive module to access Google Drive
from google.colab import drive

# ----- GPU CHECK -----

# Check whether GPU is available for PyTorch
print("GPU available:", torch.cuda.is_available())

# Print number of GPUs available
print("GPU count:", torch.cuda.device_count())

# If GPU is available, print the GPU name
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

# ----- GOOGLE DRIVE MOUNT -----

# Mount Google Drive to access large datasets stored in Drive
drive.mount('/content/drive')
```

```
GPU available: True
GPU count: 1
GPU name: Tesla T4
Mounted at /content/drive
```

```
# ----- INSTALL REQUIRED LIBRARIES -----
# Install PyTorch and related libraries with CUDA support (for GPU acceleration)
!pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

# Install TensorFlow (deep learning framework)
!pip install tensorflow

# Install core data science libraries
!pip install pandas numpy scikit-learn

# Install OpenCV (headless version) for image processing tasks
!pip install opencv-python-headless # used for reading and processing images

# Install Librosa for audio signal processing
!pip install librosa # used for audio feature extraction

# Install Hugging Face Transformers for NLP tasks (BERT, GPT, etc.)
!pip install transformers # used for NLP and language models
```

```
# ----- GOOGLE DRIVE MOUNT (COLAB ONLY) -----  
# Import Google Colab drive module  
from google.colab import drive  
  
# Mount Google Drive to access large datasets stored in Drive  
drive.mount('/content/drive')  
  
# ----- GPU AVAILABILITY CHECK -----  
# Import PyTorch to check GPU support  
import torch  
  
# Check whether GPU is available  
print("GPU available:", torch.cuda.is_available())  
  
# If GPU is available, display the GPU name  
if torch.cuda.is_available():  
    print("GPU name:", torch.cuda.get_device_name(0))
```

```
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from httpx<=0.27.0)
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from httpx<=0.27.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<=0.27.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<=0.27.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<=0.27.0)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<=0.27.0)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<=0.27.0))
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("", force_remount=True)
GPU available: True
GPU name: Tesla T4
```

TASK Explore different Deep Learning Packages and Install them

LOADING FILES IN DIFFERENT FORMATS

```
import zipfile
import os

zip_file_path = "/content/archive (1).zip"
extract_dir = "/content/archive_extracted"

# Create the extraction directory if it doesn't exist
os.makedirs(extract_dir, exist_ok=True)

# Unzip the file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

print(f"'{zip_file_path}' unzipped to '{extract_dir}'")

# List the contents of the extracted directory
print("\nContents of extracted directory:")
for root, dirs, files in os.walk(extract_dir):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name) + "/")

'/content/archive (1).zip' unzipped to '/content/archive_extracted'

Contents of extracted directory:
/content/archive_extracted/student_lifestyle_100k.csv
```

```
import pandas as pd

# Load CSV (most common tabular format)
df = pd.read_csv("/content/archive_extracted/student_lifestyle_100k.csv")
```

```
df
```

	Student_ID	Age	Gender	Department	CGPA	Sleep_Duration	Study_Hours	Social_Media_H
0	1001	22	Female	Science	3.50	7.3	3.3	
1	1002	20	Male	Engineering	2.72	5.5	7.2	
2	1003	20	Male	Medical	3.01	5.4	2.3	
3	1004	21	Male	Engineering	3.63	8.1	2.0	
4	1005	19	Male	Arts	3.14	6.8	2.6	
...	...	...	...	...	...	...	...	...
99995	100996	24	Female	Medical	2.02	7.0	0.6	
99996	100997	24	Male	Arts	2.33	5.0	3.6	
99997	100998	24	Female	Engineering	2.23	6.5	5.8	
99998	100999	19	Male	Engineering	3.61	6.1	4.8	
99999	101000	24	Female	Medical	2.10	7.9	3.5	

100000 rows × 11 columns

Task

✓ Load TSV (tab-separated)

```
df = pd.read_csv("/content/drive/MyDrive/data/train.tsv", sep="\t")
```

Load Excel (XLS/XLSX)

```
df = pd.read_excel("/content/drive/MyDrive/data/train.xlsx", sheet_name="Sheet1")
```

```
import pandas as pd
import os
```

```
extract_dir = "/content/archive_3_extracted"
```

```
df = pd.read_csv(os.path.join(extract_dir, 'movie_titles_metadata.tsv'), sep='\t', header=None)
print("Head of the DataFrame after loading:")
print(df.head())
```

Head of the DataFrame after loading:

```

0      1      2      3      4  \
0  m0  10 things i hate about you  1999  6.9  62847.0
1  m1  1492: conquest of paradise  1992  6.2  10421.0
2  m2                15 minutes  2001  6.1  25854.0
3  m3      2001: a space odyssey  1968  8.4  163227.0
4  m4                48 hrs.  1982  6.9  22289.0

0      5
0      ['comedy' 'romance']
1  ['adventure' 'biography' 'drama' 'history']
2      ['action' 'crime' 'drama' 'thriller']
3      ['adventure' 'mystery' 'sci-fi']
4  ['action' 'comedy' 'crime' 'drama' 'thriller']
```

Start coding or [generate](#) with AI.

```
import pandas as pd

# Load the uploaded Sample-Sales-Data.xlsx file into a DataFrame
df_sales = pd.read_excel("/content/Sample-Sales-Data.xlsx")

# Display the head of the DataFrame to confirm successful loading
print("Head of Sample Sales Data:")
print(df_sales.head())
```

Head of Sample Sales Data:

	Postcode	Sales_Rep_ID	Sales_Rep_Name	Year	Value
0	2121	456	Jane	2011	84219.497311
1	2092	789	Ashish	2012	28322.192268
2	2128	456	Jane	2013	81878.997241
3	2073	123	John	2011	44491.142121
4	2134	789	Ashish	2012	71837.720959

```
import zipfile
import os

zip_file_path = "/content/archive (2).zip"
extract_dir = "/content/new_archive_extracted"

# Create the extraction directory if it doesn't exist
os.makedirs(extract_dir, exist_ok=True)

# Unzip the file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

print(f'{zip_file_path} unzipped to '{extract_dir}''')

# List the contents of the extracted directory
print("\nContents of newly extracted directory:")
for root, dirs, files in os.walk(extract_dir):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name) + "/")

'/content/archive (2).zip' unzipped to '/content/new_archive_extracted'

Contents of newly extracted directory:
/content/new_archive_extracted/json_award.json
/content/new_archive_extracted/json_laureates.json
/content/new_archive_extracted/complete.csv
```

```
import pandas as pd

# Load json_award.json from the extracted archive
df_awards = pd.read_json("/content/new_archive_extracted/json_award.json")

# Display the first few rows of the DataFrame
print("Head of df_awards:")
print(df_awards.head())
```

Head of df_awards:

	awardYear	category \
0	1901	{'en': 'Chemistry', 'no': 'Kjemi', 'se': 'Kemi'}
1	1901	{'en': 'Literature', 'no': 'Litteratur', 'se': '...
2	1901	{'en': 'Peace', 'no': 'Fred', 'se': 'Fred'}
3	1901	{'en': 'Physics', 'no': 'Fysikk', 'se': 'Fysik'}
4	1901	{'en': 'Physiology or Medicine', 'no': 'Fysiol...

categoryFullName prizeAmount \

```

0 {'en': 'The Nobel Prize in Chemistry', 'no': '...' 150782
1 {'en': 'The Nobel Prize in Literature', 'no': '...' 150782
2 {'en': 'The Nobel Peace Prize', 'no': 'Nobels ...' 150782
3 {'en': 'The Nobel Prize in Physics', 'no': 'No...' 150782
4 {'en': 'The Nobel Prize in Physiology or Medic... 150782

prizeAmountAdjusted links \
0 8567159 {'rel': 'nobelPrize', 'href': 'http://masterda...
1 8567159 {'rel': 'nobelPrize', 'href': 'http://masterda...
2 8567159 {'rel': 'nobelPrize', 'href': 'http://masterda...
3 8567159 {'rel': 'nobelPrize', 'href': 'http://masterda...
4 8567159 {'rel': 'nobelPrize', 'href': 'http://masterda...

laureates dateAwarded topMotivation
0 [{'id': '160', 'knownName': {'en': 'Jacobus H....' NaN NaN
1 [{'id': '569', 'knownName': {'en': 'Sully Prud... NaN NaN
2 [{'id': '462', 'knownName': {'en': 'Henry Duna... 1901-12-10 NaN
3 [{'id': '1', 'knownName': {'en': 'Wilhelm Conr... NaN NaN
4 [{'id': '293', 'knownName': {'en': 'Emil von B... NaN NaN

```

If your JSON file is in Google Drive, you need to ensure the path you're providing is correct. Make sure Google Drive is mounted (`drive.mount('/content/drive')`) and then specify the full path, for example: `/content/drive/MyDrive/path/to/your_file.json`.

df

	awardYear	category	categoryFullName	sortOrder	portion	prizeAmount	prizeAmountAdj
0	2001	Economic Sciences	The Sveriges Riksbank Prize in Economic Scienc...	2	1/3	10000000	1229
1	1975	Physics	The Nobel Prize in Physics	1	1/3	630000	340
2	2004	Chemistry	The Nobel Prize in Chemistry	1	1/3	10000000	1176
3	1982	Chemistry	The Nobel Prize in Chemistry	1	1	1150000	310
4	1979	Physics	The Nobel Prize in Physics	2	1/3	800000	298
...	...	...	...	...	...	...	...
945	1972	Physics	The Nobel Prize in Physics	1	1/3	480000	334
946	1954	Chemistry	The Nobel Prize in Chemistry	1	1	181647	264
947	1911	Chemistry	The Nobel Prize in Chemistry	1	1	140695	732
948	1981	Peace	The Nobel Peace Prize	1	1	1000000	292
949	1963	Peace	The Nobel Peace Prize	1	1/2	265000	283

950 rows × 52 columns

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
df = pd.read_csv(url)
```

df

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
```

```
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

TASK Upload built-in dataset

```
# Import OpenCV library (used for image processing; not directly used here)
import cv2

# Import PIL to load images in Python
from PIL import Image

# Import PyTorch library
import torch

# Import matplotlib for displaying images
import matplotlib.pyplot as plt

# Import torchvision transforms for image preprocessing
from torchvision import transforms

# -----
# STEP 1: Load the image using PIL
# -----

# Open the image file from the given path
pil_img = Image.open("CAT.jpg")

# -----
# STEP 2: Define image transformations
# -----

# Resize image to 224x224 (standard input size for CNNs)
```



```
# Convert image to PyTorch tensor (values scaled to [0, 1])
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

# -----
# STEP 3: Apply transforms to the image
# -----

# Apply preprocessing and convert PIL image to tensor
# Shape becomes: [3, 224, 224] → (Channels, Height, Width)
img_tensor = transform(pil_img)

# Add batch dimension required by deep learning models
# Final shape: [1, 3, 224, 224]
img_tensor = img_tensor.unsqueeze(0)

# -----
# STEP 4: Visualize the image
# -----

# Create a figure with fixed size
plt.figure(figsize=(6, 6))

# Remove batch dimension: [3, 224, 224]
# Change tensor order from (C, H, W) to (H, W, C)
# Convert tensor to NumPy array for matplotlib
img_np = img_tensor[0].permute(1, 2, 0).numpy()

# Display the image
plt.imshow(img_np)

# Hide axis values for cleaner visualization
plt.axis("off")

# Set title for the image
plt.title("Loaded image as tensor")

# Show the image
plt.show()
```

Loaded image as tensor



```
# Import PIL to load images in Python
from PIL import Image

# Open the image file from the given path
pil_img = Image.open("/content/cat.jfif")

print("Image loaded successfully.")
```

Image loaded successfully.

```
# Import librosa library for audio processing
import librosa

# -----
# STEP 1: Load the audio file
# -----

# Load the audio file from the given path
# audio → 1D NumPy array containing the audio signal (amplitude values)
# sr → sample rate of the audio (samples per second)
# sr=None ensures the original sample rate is preserved
audio, sr = librosa.load("/content/war-drum-loop-103870.mp3", sr=None)

# -----
# STEP 2: Display basic audio information
# -----

# Print the sample rate of the audio file
print("Sample rate:", sr)

# Print the shape of the audio array
# Length indicates number of audio samples
print("Audio shape:", audio.shape)
```

Sample rate: 24000
Audio shape: (169920,)

```
# Load entire text file
with open("/content/sample.txt", "r", encoding="utf-8") as f:
    text = f.read()

# Load line-by-line
with open("/content/sample.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()
```

text

'The think tank of China's People's Liberation Army Rocket Force recently recruited 13 Chinese technicians\n from private companies, PLA Daily reported on Saturday.\nZhang Hao and 12 other science and technology experts received letters of appointment at the founding ceremony of\nthe PLA Rocket Force national defense science and technology experts panel, according to a report published by the\nPLA Daily on Saturday.\nHonored as "rocket force science and technology experts," Zhang and his fellow experts from private companies will\nserve as members of the PLA Rocket Force think tank, which will conduct research into fields like overall design of\nthe missiles, missile launching and network system technology for five years.\nThe experts will enjoy the same treatment as their counterparts from State-owned firms, the report said.\nThe PLA Daily said that this marks a new development in deepening military-civilian integration in China, which\ncould make science and technology innovation better.'

```
# -----
# SINGLE NEURON: FORWARD PASS ONLY
# Input → Weights → Sum → Activation → Output
# -----

import numpy as np

# Step 1: Input values
inputs = np.array([1, 2])      # x1 = 1, x2 = 2

# Step 2: Weights
weights = np.array([0.4, 0.6]) # w1, w2

# Step 3: Bias
bias = 0.2

# Step 4: Weighted sum
weighted_sum = np.dot(inputs, weights) + bias

# Step 5: Activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Step 6: Output
output = sigmoid(weighted_sum)

# Step 7: Print each stage
print("Input:", inputs)
print("Weights:", weights)
print("Weighted Sum (z):", weighted_sum)
print("Final Output:", output)
```

```
Input: [1 2]
Weights: [0.4 0.6]
Weighted Sum (z): 1.8
Final Output: 0.8581489350995123
```

```
# -----
# BASIC NEURAL NETWORK IMPLEMENTATION
# -----

# Step 1: Import required library
import numpy as np

# Step 2: Define input data (AND logic)
# Each row is one input sample
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
```

```

# Step 3: Define expected output
# AND logic output
y = np.array([[0],
              [0],
              [0],
              [1]])

# Step 4: Initialize weights and bias randomly
# 2 input neurons -> 1 output neuron
weights = np.random.rand(2, 1)
bias = np.random.rand(1)

# Step 5: Define activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Step 6: Define derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Step 7: Training parameters
learning_rate = 0.1
epochs = 10000

# Step 8: Training the neural network
for epoch in range(epochs):

    # Forward propagation
    z = np.dot(X, weights) + bias          # Weighted sum
    output = sigmoid(z)                   # Activation output

    # Calculate error
    error = y - output

    # Backpropagation
    d_output = error * sigmoid_derivative(output)

    # Update weights and bias
    weights += learning_rate * np.dot(X.T, d_output)
    bias += learning_rate * np.sum(d_output)

# Step 9: Display final results
print("Final Weights:\n", weights)
print("Final Bias:\n", bias)

# Step 10: Test the network
print("\nPredicted Output after Training:")
print(output)

```

Final Weights:

```
[[5.47316373]
 [5.47316373]]
```

Final Bias:

```
[-8.30246558]
```

Predicted Output after Training:

```
[[2.47887008e-04]
 [5.57642156e-02]
 [5.57642154e-02]
 [9.33627984e-01]]
```

Task

I will extract the `/content/archive (2).zip` file into a temporary directory, identify the file type(s) within it, and then generate and insert code into cell `sduM5RKMpeW6` to load the relevant data into a pandas DataFrame. Finally, I will display the head of the DataFrame to confirm the successful data load.

✓ Extract New Zip File

Subtask:

Extract the contents of the newly provided zip file `/content/archive (2).zip` into a temporary directory.

Reasoning: To extract the contents of the zip file, I will use the `zipfile` module to open and extract the archive, and the `os` module to manage the creation of the extraction directory and list its contents.

```
import zipfile
import os

zip_file_path = "/content/archive (2).zip"
extract_dir = "/content/new_archive_extracted"

# Create the extraction directory if it doesn't exist
os.makedirs(extract_dir, exist_ok=True)

# Unzip the file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

print(f"'{zip_file_path}' unzipped to '{extract_dir}'")

# List the contents of the extracted directory
print("\nContents of newly extracted directory:")
for root, dirs, files in os.walk(extract_dir):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name) + "/")

'/content/archive (2).zip' unzipped to '/content/new_archive_extracted'

Contents of newly extracted directory:
/content/new_archive_extracted/json_award.json
/content/new_archive_extracted/json_laureates.json
/content/new_archive_extracted/complete.csv
```

```
import pandas as pd

# Load CSV (most common tabular format)
df = pd.read_csv("/content/new_archive_extracted/complete.csv")

# Display the head of the DataFrame to confirm successful load
df.head()
```

	awardYear	category	categoryFullName	sortOrder	portion	prizeAmount	prizeAmountAdjuste
0	2001	Economic Sciences	The Sveriges Riksbank Prize in Economic Scienc...	2	1/3	10000000	1229508
1	1975	Physics	The Nobel Prize in Physics	1	1/3	630000	340417
2	2004	Chemistry	The Nobel Prize in Chemistry	1	1/3	10000000	1176286
3	1982	Chemistry	The Nobel Prize in Chemistry	1	1	1150000	310251
4	1979	Physics	The Nobel Prize in Physics	2	1/3	800000	298804

5 rows × 52 columns

Final Task

Subtask:

Confirm that the data has been successfully loaded into the DataFrame in cell 14 and display the head of the DataFrame to verify its contents.

✓ Summary:

Q&A

The data has been successfully loaded into the DataFrame. The head of the DataFrame (`df`) was displayed, confirming that the data was loaded correctly and contains 5 rows and 52 columns with relevant Nobel Prize information.

Data Analysis Key Findings

- The `/content/archive (2).zip` file was successfully extracted into the `/content/new_archive_extracted` directory.
- The extracted directory contained three files: `json_award.json`, `json_laureates.json`, and `complete.csv`.
- The `complete.csv` file was identified and loaded into a pandas DataFrame named `df`.
- The DataFrame `df` was successfully loaded and its head displayed, showing 5 rows and 52 columns of Nobel Prize information.

Insights or Next Steps

- The `df` DataFrame is now available for further analysis, potentially involving data cleaning, exploration, or modeling based on the Nobel Prize information it contains.

df

	awardYear	category	categoryFullName	sortOrder	portion	prizeAmount	prizeAmountAdj
0	2001	Economic Sciences	The Sveriges Riksbank Prize in Economic Scienc...	2	1/3	10000000	1229
1	1975	Physics	The Nobel Prize in Physics	1	1/3	630000	340
2	2004	Chemistry	The Nobel Prize in Chemistry	1	1/3	10000000	1176
3	1982	Chemistry	The Nobel Prize in Chemistry	1	1	1150000	310
4	1979	Physics	The Nobel Prize in Physics	2	1/3	800000	298
...	...	...	...	...	...	...	...
945	1972	Physics	The Nobel Prize in Physics	1	1/3	480000	334
946	1954	Chemistry	The Nobel Prize in Chemistry	1	1	181647	264
947	1911	Chemistry	The Nobel Prize in Chemistry	1	1	140695	732
948	1981	Peace	The Nobel Peace Prize	1	1	1000000	292
949	1963	Peace	The Nobel Peace Prize	1	1/2	265000	282

950 rows × 52 columns

Start coding or [generate](#) with AI.

Task

Extract the contents of the `/content/archive (3).zip` file into a temporary directory, identify the primary data file, load it into a pandas DataFrame named `df`, and display the head of the DataFrame to confirm the successful data load.

```
import zipfile
import os

zip_file_path = "/content/archive (3).zip"
extract_dir = "/content/archive_3_extracted"

# Create the extraction directory if it doesn't exist
os.makedirs(extract_dir, exist_ok=True)

# Unzip the file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

print(f"'{zip_file_path}' unzipped to '{extract_dir}'")

'/content/archive (3).zip' unzipped to '/content/archive_3_extracted'
```

```
print("\nContents of newly extracted directory:")
for root, dirs, files in os.walk(extract_dir):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name) + "/")

Contents of newly extracted directory:
/content/archive_3_extracted/movie_titles_metadata.tsv
```

```
import pandas as pd

# Load the TSV file into a pandas DataFrame
df = pd.read_csv(os.path.join(extract_dir, 'movie_titles_metadata.tsv'), sep='\t', header=None)

# Display the head of the DataFrame to confirm successful loading
print("Head of the DataFrame:")
print(df.head())
```

```
Head of the DataFrame:
   0      1      2      3      4  \
0  m0  10 things i hate about you  1999  6.9  62847.0
1  m1  1492: conquest of paradise  1992  6.2  10421.0
2  m2                15 minutes  2001  6.1  25854.0
3  m3      2001: a space odyssey  1968  8.4  163227.0
4  m4                48 hrs.  1982  6.9  22289.0

   5
0      ['comedy' 'romance']
1  ['adventure' 'biography' 'drama' 'history']
2      ['action' 'crime' 'drama' 'thriller']
3      ['adventure' 'mystery' 'sci-fi']
4  ['action' 'comedy' 'crime' 'drama' 'thriller']
```

Task

Extract the contents of the `/content/archive (4).zip` file, load the contained XLSX file into a pandas DataFrame named `df` in cell `sduM5RKMpew6`, and display the head of the DataFrame.

```
import zipfile
import os

zip_file_path = "/content/archive (4).zip"
extract_dir = "/content/archive_4_extracted"

# Create the extraction directory if it doesn't exist
os.makedirs(extract_dir, exist_ok=True)

# Unzip the file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

print(f"'{zip_file_path}' unzipped to '{extract_dir}'")

# List the contents of the extracted directory
print("\nContents of newly extracted directory:")
for root, dirs, files in os.walk(extract_dir):
    for name in files:
        print(os.path.join(root, name))
    for name in dirs:
        print(os.path.join(root, name) + "/")

'/content/archive (4).zip' unzipped to '/content/archive_4_extracted'

Contents of newly extracted directory:
/content/archive_4_extracted/Cola.xlsx
```

```
import pandas as pd
import os

extract_dir = "/content/archive_4_extracted"

# Load Excel (XLSX) file
df = pd.read_excel(os.path.join(extract_dir, 'Cola.xlsx'))

# Display the head of the DataFrame to confirm successful loading
print("Head of the DataFrame after loading:")
print(df.head())
```

Head of the DataFrame after loading:

	Data provided by SimFin	Unnamed: 1	Unnamed: 2	Unnamed: 3	\
0	NaN	NaN	NaN	NaN	
1	Profit & Loss statement	NaN	NaN	NaN	
2	NaN	in million USD	FY '09	FY '10	
3	NaN	NET OPERATING REVENUES	30990	35119	
4	NaN	Cost of goods sold	11088	12693	